

本机环境检查

1. **查看当前用户和目录**: 运行 `whoami` 和 `pwd`。前者显示当前登录用户名，后者显示当前所在目录。例如输出中会出现你的用户名和主目录路径 (如 `/home/yourname`)。这说明你已在自己用户空间中操作。
2. **查看系统信息**: 运行 `uname -a` 和 `lsb_release -a`。`uname -a` 会显示内核和操作系统版本信息，`lsb_release -a` (若已安装) 会显示发行版详情。检查输出中是否提到 Ubuntu，以及版本号等信息。成功时不会报错；如果 `lsb_release` 未安装，可以忽略，但 `uname -a` 应正常显示。
3. **检查Shell 和常用命令**: 运行 `echo $SHELL` 查看使用的默认 Shell (通常是 `bash`)；运行 `which bash`、`which ssh`、`which code`、`which node`、`which npm`、`which python3`、`which pip3`。这会显示这些命令的可执行路径 (如果存在)。比如 `which python3` 应该输出 `/usr/bin/python3` 或类似路径。如果某个命令输出为空，说明该命令未安装或不在 PATH 中 (根据需求再做其他方案)。

网络连通性测试

- 运行 `ping -c 3 8.8.8.8` 以测试与外网的 IP 连通性 (Google DNS)。如果成功，会看到类似 “3 packets transmitted, 3 received” 的信息；如果看到 “100% packet loss” 或需要权限提示 (icmp 被禁用)，说明 ICMP ping 受限，可尝试下一步。
- 运行 `ping -c 3 github.com` 以测试 DNS 解析和与 GitHub 通信。正常情况下会显示来自 github.com 的 ping 响应 (如果被禁止，可能超时或被拦截)。
- 如果 ping 被系统禁止 (提示 Operation not permitted 等)，改用 HTTP 请求检查连通性：运行 `curl -I https://github.com` 和 `curl -I https://api.openai.com`。这会输出 HTTP 响应头 (例如 HTTP/1.1 200 OK)。如果看到 200 或类似成功状态，说明网络正常。如果 `curl` 也不可用 (`command not found`)，请继续使用已有环境做后续步骤，不要安装系统工具。

检查方法： 成功时每个命令都应正常输出信息 (如用户名、系统信息、HTTP 状态等)，且无 “command not found” 或 “Permission denied” 之类错误。

配置 Codex / 代理 (无 sudo)

1. **检查已有 Codex CLI**: 运行 `which codex` 查看是否已安装；如果找得到可执行路径，继续运行 `codex --version` 和 `codex --help`。如果 `codex --version` 能显示版本号，表示已安装成功，可以直接运行 `codex` 启动工具。如果上述命令均失败 (未找到 codex)，继续下一步。

2. 若没有 Codex，但有 npm

: 首先在用户目录设置全局 npm 安装路径:

bash

复制

```
1 mkdir -p ~/.npm-global
2 npm config set prefix ~/.npm-global
3 echo 'export PATH="$HOME/.npm-global/bin:$PATH"' >> ~/.bashrc
4 source ~/.bashrc
```

这会将 npm 的全局安装目录改为

```
1 | ~/.npm-global
```

(避免需要 sudo 权限)。然后运行：

bash

复制

```
1 | npm install -g @openai/codex
```

根据 OpenAI 指南，通过 npm 安装 Codex CLI

。安装完成后，再次运行

```
1 | which codex
```

应显示

```
1 | ~/.npm-global/bin/codex
```

(或相似路径)，并运行

```
1 | codex --version
```

应看到版本信息。

- **检查成功：** `codex` 命令可以正常运行（会提示登录或帮助信息），且 `codex --version` 显示版本号。
- **安装失败：** 如果出现权限错误，请确认前面的 `npm config set prefix` 已生效；如果网络问题导致下载失败，检查网络设置（但不要使用 VPN）；如果出现证书或代理错误，在遵守考场规则下尝试调整（例如设置 `NODE_EXTRA_CA_CERTS` 等）。如果实在无法解决，请走下述替代方案。

3. 若无 npm/Node

：先检查是否有 Conda：运行

```
1 | which conda
```

和

```
1 | conda env list
```

。如果找得到 Conda，可使用 Conda 创建包含 Node.js 的环境：

bash

复制

```
1 | mkdir -p /vepfs/$USER/envs
2 | conda create -p /vepfs/$USER/envs/codex-node nodejs npm -y
3 | conda activate /vepfs/$USER/envs/codex-node
4 | npm install -g @openai/codex
```

这与之前类似，只是把 Node/NPM 和 Codex 安装到你自己的环境里。Conda 官方文档也支持用

```
1 | --prefix
```

指定路径创建环境

。

- **检查成功**：激活环境后，运行 `codex --version` 显示正常版本。
- **无 Conda 或安装失败**：则不再继续安装 Codex。此时使用以下替代方案：在允许范围内，使用浏览器版 ChatGPT/Codex 或本地 VS Code 编辑器等方式进行代码辅助；或者在开发机上手动复制报错和代码片段交给 AI 分析。

4. **API Key 使用**：如果 Codex 需要 API Key，**只通过临时环境变量设置**，例如 `export`

`OPENAI_API_KEY="你的key"`。千万不要将 Key 写入任何代码文件、提交记录或公开分享，保证 Key 安全。

检查方法：安装并配置后，运行 `codex --help` 或检查 `~/.codex` 目录，确认 Codex CLI 可以启动且无明显错误（参考）。

SSH 连接开发机

1. **尝试连接**：运行 `ssh dev`。这会尝试使用 `~/.ssh/config` 中 `Host dev` 的设置连接。如果连通，会显示远端主机名和欢迎信息。

2. 连接失败的排查

：如果出现错误，根据提示检查：

- 确认 `~/.ssh` 目录存在：`ls -la ~/.ssh`，应包含私钥（`id_rsa`、`id_ed25519` 等）和可能的 `config` 文件。
- 查看 SSH 客户端配置：

```
1 | cat ~/.ssh/config
```

（如有）。检查是否有名为

```
1 | dev
```

的条目以及对应的

```
1 | HostName
```

、

```
1 | User
```

、

```
1 | Port
```

。如果

```
1 | ~/.ssh/config
```

不存在但老师提供了服务器地址、用户名和端口，可新建：

bash

复制

```
1 | mkdir -p ~/.ssh
2 | nano ~/.ssh/config
```

按格式填写（示例）：

pgsql

复制

```
1 | Host dev
2 |     HostName your.server.address
3 |     User your_username
4 |     Port 22
```

保存后运行

```
1 | chmod 600 ~/.ssh/config
```

- 。
- 检查权限：SSH 要求私钥和目录权限严格。可运行：

bash

复制

```
1 | chmod 700 ~/.ssh
2 | chmod 600 ~/.ssh/config
3 | chmod 600 ~/.ssh/id_rsa 2>/dev/null
4 | chmod 600 ~/.ssh/id_ed25519 2>/dev/null
```

这确保只有自己可以读写 SSH 相关文件。

- 使用详细模式查看错误：`ssh -v dev`，观察输出信息中的错误提示（如认证失败、连接超时等）。根据提示逐步修复（如询问监考老师服务器地址）。**注意：**不要泄露或拷贝私钥内容，也不要私钥贴给任何人。

检查方法：成功连接后，你会看到类似 `welcome to ...` 的远程主机提示符，并且执行 `whoami` 会显示远程用户名。如果仍有问题，根据错误消息和上述检查项逐一排查。

登录开发机后的检查

连接成功后，你会获得远端的命令行。首先执行基础检查命令：

bash

复制

```
1 | whoami
2 | hostname
3 | pwd
4 | nvidia-smi
5 | python3 --version
6 | which python3
7 | which conda
8 | conda env list
9 | df -h
10 | ls -lah /vepfs-readonly/
11 | ls -lah /vepfs/
```

- 解释与成功判定

:

- `whoami`、`hostname`、`pwd` 显示当前登录用户、主机名和工作目录。确保用户名是你的帐号（\$USER），路径是远端的主目录或当前目录。
- `nvidia-smi` 应列出服务器上的 GPU 型号和使用情况，如 NVIDIA A100 80G。如果显示 GPU 列表和内存占用，说明 GPU 驱动正常。
- `python3 --version`、`which python3`、`which conda`、`conda env list` 查看 Python 版本与 Conda 环境。如果 `which conda` 有输出且列出了环境，则说明 Conda 已安装且可用。
- `df -h` 查看磁盘使用情况，确认目标路径（如 `/vepfs`）有足够空间。

- `ls -lah /vepfs-readonly/` 应列出题目目录 (problem1 等), 确认只读目录存在; `ls -lah /vepfs/` 查看你的用户目录是否存在以及权限。
- 检查 PyTorch GPU 可用性
 - : 若系统未安装

```
1 | nvcc
```

编译器, 不要慌。可以通过 Python 直接测试:

bash

复制

```
1 | python3 -c "import torch; print(torch.__version__);  
print(torch.cuda.is_available()); print(torch.cuda.get_device_name(0) if  
torch.cuda.is_available() else 'no cuda')"
```

成功情况下, 这将打印 PyTorch 版本号 (如

```
1 | 2.x.x
```

), 然后

```
1 | True
```

或

```
1 | False
```

表示 CUDA 是否可用, 最后如果可用则打印 GPU 名称 (如

```
1 | NVIDIA A100
```

)

。如果打印

```
1 | False
```

或 “no cuda”，说明当前环境未启用 GPU；如果出现

```
1 | ModuleNotFoundError
```

，说明该环境缺少 PyTorch，需要后续安装。

创建工作目录并复制题目

1. 创建可写目录

：查看你的远端用户根目录是否可写：运行

```
1 | echo $USER
```

确认用户名，尝试创建目录：

bash

复制

```
1 | mkdir -p /vepfs/$USER/research_practice
2 | cd /vepfs/$USER/research_practice
```

如果

```
1 | /vepfs/$USER
```

不可写（权限被拒绝），则使用你的家目录：

bash

复制

```
1 | mkdir -p ~/research_practice
2 | cd ~/research_practice
```

2. 复制题目文件

：将只读目录下的题目复制到新建的工作目录：

bash

复制

```
1 cp -r /vepfs-readonly/problem1 .
2 cp -r /vepfs-readonly/problem2 .
3 cp -r /vepfs-readonly/problem3 .
4 cp -r /vepfs-readonly/problem4 .
5 ls -lah
```

这将复制 problem1~4 文件夹到当前目录下。运行

```
1 ls -lah
```

应能看到

```
1 problem1
```

、

```
1 problem2
```

等目录。

3. **注意事项：** `/vepfs-readonly/` 目录是只读的**题目资源**，不能修改其内容，只能复制。工作过程中不要在 `/vepfs-readonly/` 下删除或修改文件，以免影响他人。删除命令要格外小心，优先使用 `rm -i` 交互确认模式，**绝对不要**用 `rm -rf /` 等危险操作。

检查方法： 成功时，`ls -lah` 中会列出你的 `research_practice` 目录和里面的 `problem1~4` 文件夹，表明复制成功。

配置 Python 环境（无 sudo）

根据环境状况选择以下方案：

- **方案 A：使用已有 Conda 环境。** 运行 `conda env list` 查看已有环境。若其中有符合要求的环境（包含必要的包如 PyTorch），可以直接激活：

bash

复制

```
1 conda activate 环境名
2 python --version
3 pip list | grep torch
```

如果激活后能正常运行 Python，且发现合适的包版本（例如 `torch` 列在 `pip list` 中），则跳过后续安装步骤。

- **方案 B：用户目录下新建 Conda 环境。** 如果没有合适环境或需要隔离环境：

bash

复制

```
1 mkdir -p /vepfs/$USER/envs
2 conda create -p /vepfs/$USER/envs/practice python=3.10 -y
3 conda activate /vepfs/$USER/envs/practice
```

这样会在 `/vepfs/$USER/envs/practice` 创建一个 Python 3.10 环境（无需 sudo 权限）。成功后，`python --version` 应显示 3.10，`which pip` 指向该环境。

- **方案 C：使用 venv。** 如果 Conda 不可用，可用 Python 自带的 venv：

bash

复制

```
1 cd /vepfs/$USER/research_practice
2 python3 -m venv .venv
3 source .venv/bin/activate
4 python -m pip install --upgrade pip
```

这会在当前目录创建 `.venv` 文件夹并激活虚拟环境。成功时，终端提示符前会出现 `(.venv)`；运行 `which python` 显示指向 `.venv/bin/python`。

- 如果 `python3 -m venv` 报错缺少模块，不要 `sudo apt install python3-venv`，改用方案 B 或系统 Python（确保 Python3 已安装）。

- **方案 D：pip 用户目录安装。** 若仅需安装单个包，可用：

bash

复制

```
1 python3 -m pip install --user <包名>
```

这样包会安装到 `~/.local/lib/python...` 中。并确保把 `~/.local/bin` 添加到 PATH：

bash

复制

```
1 echo 'export PATH="$HOME/.local/bin:$PATH"' >> ~/.bashrc
2 source ~/.bashrc
```

安装依赖：如果题目给了 `requirements.txt`，尝试：

bash

复制

```
1 python3 -m pip install -r requirements.txt
```

如果因权限问题失败，可加 `--user`：

bash

复制

```
1 | python3 -m pip install --user -r requirements.txt
```

注意：遇到网络问题不可随意使用 VPN。在无网络或不便的情况下，先查看当前环境是否已包含必需包，或者题目目录是否提供了轮子或离线包；如有问题可询问监考老师是否允许提供额外离线依赖。

检查方法：安装/创建完毕后，运行 `python -c "import torch; print(torch.__version__)"` 等检查主要库是否可用，无报错即表成功。

VS Code 远程开发（无 sudo）

1. **使用 VS Code Remote-SSH：**在本地 VS Code 中打开（无需 sudo），确保安装了 **Remote - SSH** 插件（扩展市场搜索“Remote - SSH”安装）。在 VS Code 中按 `F1`，输入并选择 `Remote-SSH: Connect to Host...`，然后选择 `dev`（对应 `~/.ssh/config` 中的配置）。连接成功后，VS Code 会在新窗口打开远端资源。打开文件资源管理器，定位到 `/vepfs/$USER/research_practice` 并展开，可以直接在 VS Code 中编辑和保存远端文件。

2. Remote-SSH 失败的替代方案

：若 Remote-SSH 无法使用：

- **方案 A：**直接在远端终端编辑文件。使用 `nano` 文件名 或 `vim` 文件名 进行修改并保存。

- 方案 B

：本地编辑后上传。可以本地用编辑器改好文件，然后通过

```
1 | scp
```

上传：

bash

复制

```
1 | scp 本地文件 dev:/vepfs/$USER/research_practice/
```

- 方案 C

：使用

```
1 | rsync
```

同步目录：

bash

复制

```
1 | rsync -avP 本地目录/ dev:/vepfs/$USER/research_practice/problemX/
```

如果目标服务器没有

```
1 | rsync
```

, 可使用

```
1 | scp -r
```

整个目录。

检查方法：在 VS Code 的远端终端运行 `ls` 或编辑文件确保文件已更新；在本地同步后可再次登录 dev 检查同步结果。

常用 Linux 命令参考

- 文件与目录操作

:

- `pwd`: 显示当前目录路径;
- `ls -lah`: 列出当前目录详细文件列表 (包括隐藏文件和人类可读大小);
- `cd 目录`: 进入指定目录 (如 `cd ~` 返回家目录);
- `mkdir -p 目录`: 创建目录 (包括上级, 不存在则创建);
- `cp 文件 目标`: 复制文件; `cp -r 源目录 目标`: 递归复制目录;
- `mv 源 目标`: 移动/重命名文件或目录;
- `rm -i 文件`: 交互式删除文件 (提示确认)。

- 查看文件内容

:

- `cat 文件`: 快速查看文件内容 (一次输出全部);
- `less 文件`: 分页查看文件; 可用 `q` 退出;
- `head 文件`: 查看文件开头, 默认前 10 行;
- `tail 文件`: 查看文件末尾, 默认后 10 行; `tail -f 日志文件`: 动态追踪文件更新 (常用于日志)。

- 搜索与查找

:

- `grep -R "关键词" .`: 递归搜索当前目录下包含关键词的行;
- `find . -name "*.py"`: 查找当前目录及子目录下所有 `.py` 文件;

- 磁盘与空间

:

- `df -h`: 查看各分区磁盘使用情况;
- `du -sh *`: 查看当前目录下各文件/目录大小汇总。

- 权限设置

:

- `chmod +x script.sh`: 赋予 script.sh 执行权限;

- 压缩与解压

:

- `tar -czf 压缩包名.tar.gz 目录/`: 将目录压缩为 gzip 格式;
- `unzip 文件.zip`: 解压 zip 文件;

- 进程管理

:

- `ps aux | grep python`: 查看所有 Python 进程;
- `top`: 实时查看系统资源占用; `kill PID`: 终止指定进程号;

- GPU 监控

:

- `nvidia-smi`: 实时查看 GPU 状况; `watch -n 1 nvidia-smi`: 每秒刷新显示;

- 后台运行与任务管理

:

- 使用

```
1 | nohup
```

(no hang up) 让进程在退出终端后继续运行

:

bash

复制

```
1 | nohup python train.py > train.log 2>&1 &
```

这会将输出写入

```
1 | train.log
```

并放到后台运行（退出终端后依然执行）。可用

```
1 | jobs
```

查看后台任务，或

```
1 | ps aux
```

查进程。

- **tmux** (如果可用) : `tmux new -s train` 创建新会话，运行任务；`tmux attach -t train` 恢复会话；在 tmux 内按 `Ctrl+B` 后按 `D` 可分离会话，不影响任务运行。如果无 tmux，使用上述 `nohup`。

检查方法：运行 `nohup` 后系统通常提示 `appending output to nohup.out`。使用 `ps aux` 或 `top` 确认对应的 Python 进程在运行。

训练与日志管理

- 单题分步操作

: 每个

```
1 | problemX
```

单独处理，不要混淆。进入问题目录前先查看内容：

bash

复制

```
1 | find . -maxdepth 3 -type f | sort
2 | find . -iname "*readme*" -o -iname "*.md" -o -iname "*.txt"
```

这两条命令列出当前问题下所有文件和 README 文档，先了解题目要求。

- **最小样本调试：**如题目或代码提供了小样例，先跑小样例验证流程正确，再进行全量训练，避免一次耗时过久。
- 启动训练并保存日志
: 在问题目录中运行训练脚本时，建议保存日志：

bash

复制

```
1 | python train.py 2>&1 | tee train.log
```

这样会边运行边在屏幕显示，并写入

```
1 | train.log
```

。如果任务很长，可放到后台：

bash

复制

```
1 | nohup python train.py > train.log 2>&1 &
```

然后关闭终端也不影响训练。用

```
1 | tail -f train.log
```

实时查看最新日志输出。

- **监控 GPU**：训练时持续运行 `nvidia-smi` 或 `watch -n 1 nvidia-smi`，查看显存和 GPU 利用率。
- **显存不足应对**：若遇到 CUDA OOM（显存不足），可优先尝试降低训练需求：减小 batch size、降低 `num_workers`、使用半精度（fp16/bf16）、缩短输入序列长度、换用更小模型等。确保没有其他任务占用同一 GPU。只能终止自己启动的任务（如用 `kill` 杀掉自己的进程），不要结束他人进程。

检查方法：训练中，`train.log` 应不断更新输出；GPU 监控显示显存占用上升；任务完成后可以看到训练结束的提示或性能指标。

提交前检查

- **阅读提交要求**：仔细查看每道题目的“提交内容”说明，确认需要哪些文件（例如：预测结果、模型 checkpoint、代码、README 等）以及文件名格式是否固定。确保**不提交多余的大文件**（如缓存数据、大型中间文件）。

- 检查文件列表

：使用以下命令查看最终目录结构和大小：

bash

复制

```
1 | find . -maxdepth 3 -type f | sort
2 | du -sh *
```

确认只包含需要提交的文件和文件夹。

- 打包提交

：按照要求使用 tar 压缩（示例）：

bash

复制

```
1 tar -czf submit_problem1.tar.gz problem1/
2 tar -czf submit_problem2.tar.gz problem2/
```

压缩后再检查压缩包内容：

bash

复制

```
1 ls -lh *.tar.gz
2 tar -tzf submit_problem1.tar.gz | head
```

确认打包内容正确、文件名符合要求。

检查方法：提交包解压后，应能看到包含正确文件路径和文件，且大小合理。最终提交时确保不含 `.cache`、`.venv` 等无关目录。

遇到问题时

如果在上述任何步骤遇到错误，请先**贴出终端输出信息**给助教。助教会根据报错提示判断问题类型并给出安全的下一步建议。请务必遵守：不使用 `sudo`、不重装系统级软件、不删除不明文件、不暴露任何密钥或凭证。按照错误信息分析问题，再进行修复。

总之，每步操作后请根据终端输出核对成功与否，并谨慎处理错误。以上指导基于常见经验和官方文档编写，确保在无 `sudo` 环境下使用已有资源完成实验。祝你实训顺利！